

SKILL: taller-bv-dashboard

Instrucciones principales para actualizar el Dashboard Taller BV.

Contexto del Proyecto

- **Proyecto:** Taller BV — Electroestrada
- **Dashboard:** C:\Users\Estrada\Desktop\Claude\Reportes\Taller BV\Dashboard_Pedidos.html
- **Google Sheet fileId:** 1i09pLENwSmQAv9VLJsmz-nQsa88NqdVJ (exportar como XLSX)
- **GitHub Pages:** https://Erik07-EE.github.io/Dashboard-Taller-BV/Dashboard_Pedidos.html
- **Script subir:** Subir_a_GitHub.bat (doble clic en Explorer)
- **Git user:** erik@electroestrada.com.ar / Erik07-EE
- **Skills:** taller-bv-dashboard (esta) + taller-bv-dashboard-tech (código exacto)

Estructura del XLSX

El sheet descargado es XLSX binario (no CSV). Hojas relevantes: OF-IND, OF-ROT, OF-EST, Urgentes.

Leer con `openpyxl.load_workbook(path, data_only=True)`.

- **Fila 1:** objetivos. Buscar la celda objetivo <Mes> (del mes en curso, detectado por fecha AR) y obj. S.1...obj. S.5. El valor mensual está en col+1. Los semanales van ANTES del label mensual (entre el objetivo del mes anterior y el de este mes).
- **Detección de mes automática:** sincronizar el **mes en curso** + cualquier ****mes futuro** ya armado** en el sheet (ej. agosto cuando aparecen sus columnas). NO hardcodear el mes. Los meses históricos ya congelados en el dashboard NO se re-sincronizan.
- **Mes armado sin objetivo cargado:** incluirlo igual como bloque vacío (`obj_mensual=0`, `semanas=[]`, `codigos=[]`); un sync posterior lo completa cuando se carguen los números.
- **Fila 2:** headers con fechas datetime (cada columna = un día hábil de producción).
- **Filas 3+:** datos por código. Excluir filas donde el código es "-" o vacío (son totales).
- **Estatores:** ignorar la columna con fecha 30/04.
- **Obj. semanales** del sheet vienen decimales → redondear al entero más cercano en OF_DATA.
- **Códigos con coma decimal** (ej. IB2903,10) → normalizar a punto (IB2903.10).

Columnas de la hoja Urgentes

- Col · Campo
- 1 · N
- 2 · Ingreso
- 3 · GA
- 4 · Código
- 5 · Condición
- 6 · Cliente
- 7 · Días obj.
- 8 · Entrega sol.
- 12 · Reclamo
- 13 · Observación
- 14 · Entrega real
- 16 · Días prod. (P)
- 17 · Demora (Q)

Flujo Obligatorio (respetar siempre este orden)

1. Descargar el Google Sheet con `download_file_content`, `fileId 1i09pLENWSmQAv9VLJsmz-nQsa88NqdVJ`. Si es archivo XLSX nativo en Drive se baja directo; si es Google Sheet nativo usar `exportMimeType application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`.
2. Decodificar base64 y guardar como `/tmp/urgentes.xlsx` (verificar header PK).
3. Leer el estado actual del dashboard (OF_DATA y PEDIDOS con Node.js).
4. Extraer producción + pedidos del XLSX con `openpyxl` (ver skill técnica; detección de mes automática).
5. **Mostrar PREVIEW al usuario — ESPERAR OK antes de continuar.**
6. Aplicar cambios al HTML (string replacement quirúrgico con Python/Node).
7. Actualizar el badge con `re.sub(r"const _ULTIMO_SYNC='[^']*'", f"const _ULTIMO_SYNC='{fecha}'", html)`.
8. Validar con Node.js (todos los scripts deben pasar sin error).
9. Pedirle al usuario que ejecute `Subir_a_GitHub.bat`.
10. Verificar que el badge en GitHub Pages coincida con el local.

Formato del PREVIEW (obligatorio antes de aplicar)

```
PREVIEW — Cambios detectados [fecha]
=====
OBJETIVOS MENSUALES
GA          Dashboard   Sheet   Estado
Inducidos   XXX          XXX      OK/DIFF
PRODUCCION [MES]
GA          Actual   Nuevo   S1obj S1prod S2obj S2prod ...
CODIGOS NUEVOS/ACTUALIZADOS
Inducidos: IB2311.20(B,+3)...
PEDIDOS ACTUALIZADOS (X cambios)
n=XX [codigo] campo: viejo -> nuevo
PEDIDOS NUEVOS
n=XX [ga] [codigo] [condicion] sol:[fecha]
=====
¿Aplicar estos cambios? (OK / ajustes)
```

Reglas Críticas

- NUNCA aplicar sin PREVIEW confirmado.
- NUNCA asumir objetivos — leer de la fila 1 de cada hoja OF-*
- NUNCA calcular objetivos semanales — leerlos del sheet y redondear al entero.
- NUNCA contar filas con código "-" (duplican totales).
- NUNCA incluir la columna 30/04 en Estatores.
- NUNCA sobrescribir datos existentes del dashboard con vacíos del sheet.
- NUNCA usar `html.find('';')` para el badge — usar `re.sub` con regex.
- NUNCA re-sincronizar meses históricos ya congelados; solo mes en curso + futuros armados.
- NUNCA omitir la validación Node.js post-cambios.
- Normalizar coma→punto en los códigos de pedidos.
- `esSiCambiar` = pedidos con reclamo que contiene "cambiar" → excluir de métricas.
- Los objetivos se revalidan cada mes — siempre pueden cambiar.
- `dias_prod` negativo en el sheet = artifact de fórmula → guardar como null.
- **Sincronizar la carpeta del proyecto** (`Desktop\Claude\Reportes\Taller BV`) cada vez que se actualice memoria, prompt o skills: reflejar el cambio en `Prompt/Instrucciones_Proyecto.docx`, `skills/` y regenerar sus PDF.

Métricas (cómo se calculan)

- **IMPORTANTE (29/07/2026):** Días prom. y % prom. se calculan SOLO sobre pedidos ENTREGADOS

(con fecha de entrega real). Los pedidos en proceso NO cuentan; si un GA/condición no tiene entregados, se muestra "—". Evita mostrar adelanto/retraso de artículos aún en fabricación.

Helpers en el HTML: `entregadoP(p)` y `dpProm(arr)`.

- **Días prod. prom.:** promedio SIMPLE de `dias_prod` de los pedidos ENTREGADOS (sin Si-Cambiar).
- **% prom. (tarjeta General y solapas por GA):** $(\text{días prom} - 3) / 3$ con el promedio de días SIN redondear (solo entregados). Verde = adelanto, rojo = retraso. (Criterio del 23/07/2026.)
- **avgDemDP** (usado en % por condición): $\text{avg}((\text{dias_prod} - \text{dias_objetivo}) / \text{dias_objetivo})$ para pedidos ENTREGADOS con `dias_prod != null` — desvío de cada pedido contra su propio objetivo.
- **% Servicio:** `avgDem` = promedio de la demora efectiva (incluye estimados de pedidos vencidos sin fecha real). Basado en la demora de entrega (columna Q del sheet).
- **Lupa % Servicio:** muestra por pedido — Sector / Código / Días prod. (col P) / % Servicio (col Q).
- **Cálculo demora:** $\text{dias_habiles}(\text{entrega_sol}, \text{entrega_real}) / \text{dias_objetivo}$. Negativo = adelanto, positivo = retraso. Solo días hábiles (lunes a viernes).